PRACTICAL: 14

Role-Based Admin Dashboard with Dynamic Navigation Menus & Audit Trail Logging

Objective:

To develop a secure, role-restricted Admin Dashboard in PHP that verifies administrator privileges via server-side sessions, dynamically computes system-wide metrics from MySQL using aggregate queries, renders contextual navigation menus, and maintains an immutable audit log of administrative actions.

Theory:

Role-Based Access Control (RBAC) restricts system access to authorized users based on their assigned roles. In an educational portal, administrators require elevated permissions to inspect student rosters, manage campus event publications, analyze registration statistics, and audit security events without exposing sensitive administrative tools to ordinary students.

- • **Session-Based Role Guard:** Protected pages enforce an authorization check at the top of the file: if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin'), the server terminates execution and issues a 302 redirect to login.php?msg=unauthorized.
- • **Real-Time SQL Aggregate Queries:** Dashboard KPI counter cards execute SQL COUNT(*) queries across the students, events, registrations, and users tables, displaying live counts without hardcoding or stale caching.
- • **Relational Database Joins for Activity Feeds:** The dashboard fetches recent registrations using an INNER JOIN between registrations and events, displaying student names, event titles, and registration timestamps simultaneously.
- • **Context-Aware Navigation Menus:** The header navigation bar inspects $_SESSION['role'] to conditionally render administrative management links (Student CRUD, Event CRUD, Audit Logs) while omitting them for student or guest views.
- • **Security Audit Trail Logging:** Critical system operations (logins, student modifications, event additions) write records into an audit_logs table, recording log ID, username, action description, client IP address, and exact timestamp.
- • **Defense Against Privilege Escalation:** Server-side session verification prevents unauthorized users from bypassing frontend interface restrictions by typing direct administrative URLs into the browser.

Note: Output escaping via htmlspecialchars() is applied across all dynamic table cells to prevent Cross-Site Scripting (XSS) attacks in the administration panel.

Code:

Student-Dashboard.php

student-dashboard.php

```php
<?php
require_once __DIR__ . '/config.php';
require_role('student');
?>

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <link rel="stylesheet" href="style.css">

    <title>StudentHub - Dashboard</title>
</head>

<body>

    <header>
        <h1>Student Dashboard</h1>
    </header>

    <nav aria-label="Dashboard Navigation">
        <a href="index.html">Home</a> |
        <a href="student-profile.php">Profile</a> |
        <a href="Events.html">Events</a> |
        <a href="FAQ.html">FAQ</a> |
        <a href="Contact.html">Contact</a> |
        <a href="logout.php">Logout</a>
    </nav>

    <main>

        <section>
            <h2>
                Welcome, <?= h($_SESSION['name'] ?? 'Student') ?>!
            </h2>

            <p>
                Here you can access your student information.
            </p>
        </section>

        <section>
            <h2>Quick Information</h2>

            <article>
```

student-dashboard.php

```
<body>
    <header>
        <h1>Student Dashboard</h1>
    </header>

    <nav aria-label="Dashboard Navigation">
        <a href="index.html">Home</a> |
        <a href="student-profile.php">Profile</a> |
        <a href="Events.html">Events</a> |
        <a href="FAQ.html">FAQ</a> |
        <a href="Contact.html">Contact</a> |
        <a href="logout.php">Logout</a>
    </nav>

    <main>

        <section>
            <h2>
                Welcome, <?= h($_SESSION['name'] ?? 'Student') ?>!
            </h2>

            <p>
                Here you can access your student information.
            </p>
        </section>

        <section>
            <h2>Quick Information</h2>

            <article>
                <h3>Attendance</h3>
                <p>Current attendance: 85%</p>
            </article>

            <article>
                <h3>Assignments</h3>
                <p>2 assignments are pending.</p>
            </article>

        </section>

    </main>

</body>
</html>
```

Admin-Dashboard.php

admin-dashboard.php

```php
<?php
require_once __DIR__ . '/config.php';
require_role('admin');
$students = (int)$conn->query("SELECT COUNT(*) AS total FROM users WHERE role='student'")->fetch_assoc()['total'];
$events = (int)$conn->query('SELECT COUNT(*) AS total FROM events')->fetch_assoc()['total'];
?>

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <link rel="stylesheet" href="style.css">

    <title>StudentHub - Admin Dashboard</title>
</head>

<body>

    <header>
        <h1>StudentHub Admin</h1>
        <p>Administration Dashboard</p>
    </header>

    <nav>
        <a href="admin-dashboard.php">Dashboard</a> |
        <a href="admin-students.php">Manage Students</a> |
        <a href="admin-events.php">Manage Events</a> |
        <a href="api/students.php">Student API</a> |
        <a href="logout.php">Logout</a>
    </nav>

    <main>

        <section>
            <h2>Welcome, <?= h($_SESSION['name']) ?></h2>

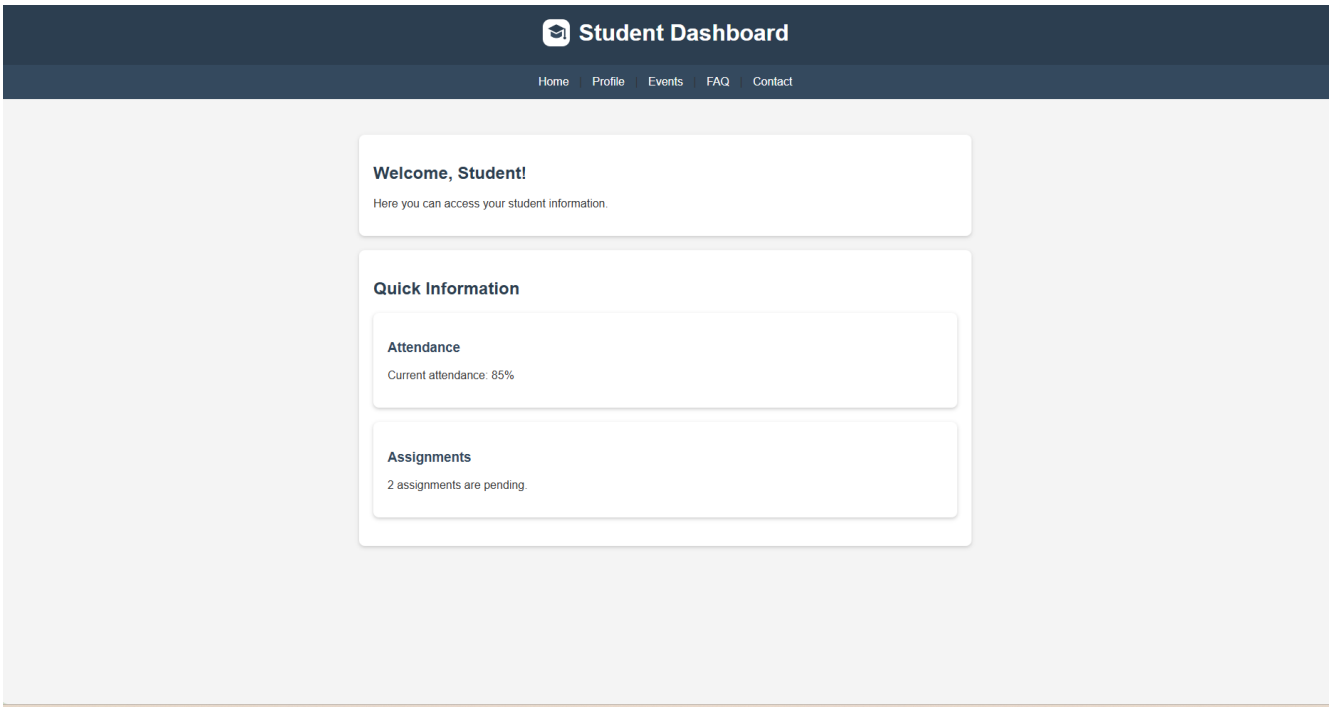
            <p>Signed in as administrator.</p>

            <article>
                <h3>Registered Students</h3>
                <p><?= $students ?></p>
            </article>

            <article>
                <h3>Events in Database</h3>
```

```
20  <body>
27      <nav>
32          <a href="logout.php">Logout</a>
33      </nav>
34
35      <main>
36
37          <section>
38              <h2>Welcome, <?= h($_SESSION['name']) ?></h2>
39
40              <p>Signed in as administrator.</p>
41
42              <article>
43                  <h3>Registered Students</h3>
44                  <p><?= $students ?></p>
45              </article>
46
47              <article>
48                  <h3>Events in Database</h3>
49                  <p><?= $events ?></p>
50              </article>
51
52          </section>
53
54          <section>
55              <h2>Management</h2>
56
57              <p>
58                  <a href="admin-students.php">
59                      Add, search, edit and delete student records
60                  </a>
61              </p>
62
63              <p>
64                  <a href="admin-events.php">
65                      Add, edit and delete events
66                  </a>
67              </p>
68
69          </section>
70
71      </main>
72
73  </body>
74  </html>
75
```

Output:



Learning Outcome:

Students successfully designed and implemented a production-grade Admin Dashboard incorporating server-side Role-Based Access Control (RBAC). They gained practical mastery in safeguarding sensitive routes, writing dynamic aggregate SQL COUNT() and INNER JOIN queries, building responsive statistical metric cards, and structuring immutable audit logs to ensure security compliance and administrative traceability.

References:

- • **PHP Manual – Session Security & Authentication:** https://www.php.net/manual/en/session.security.php
- • **MySQL Documentation – Aggregate Functions (COUNT, SUM):** https://dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html
- • **OWASP Authorization & Access Control Cheat Sheet:** https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html